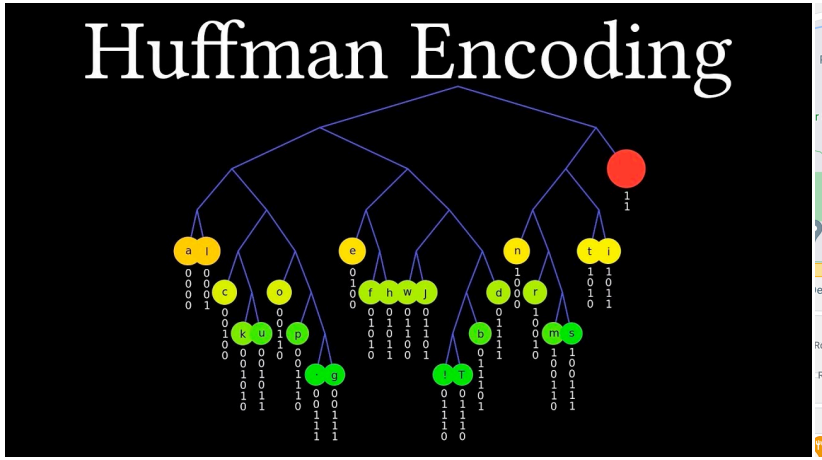
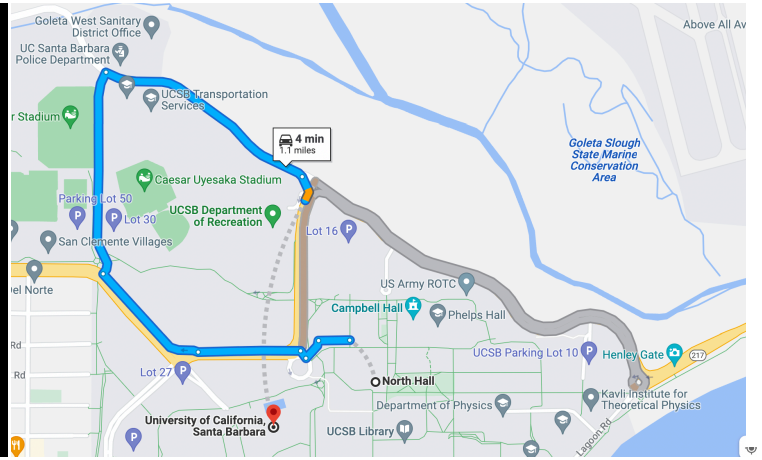
Lecture handout
https://bit.ly/CS24-binaryheap

BINARY HEAP

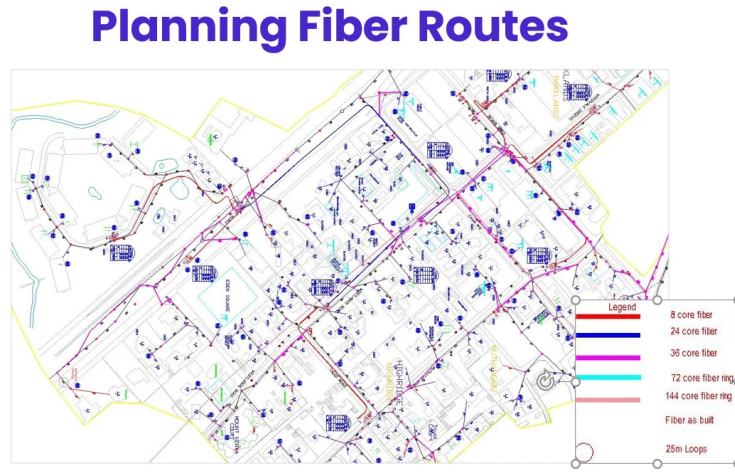
Problem Solving with Computers-II



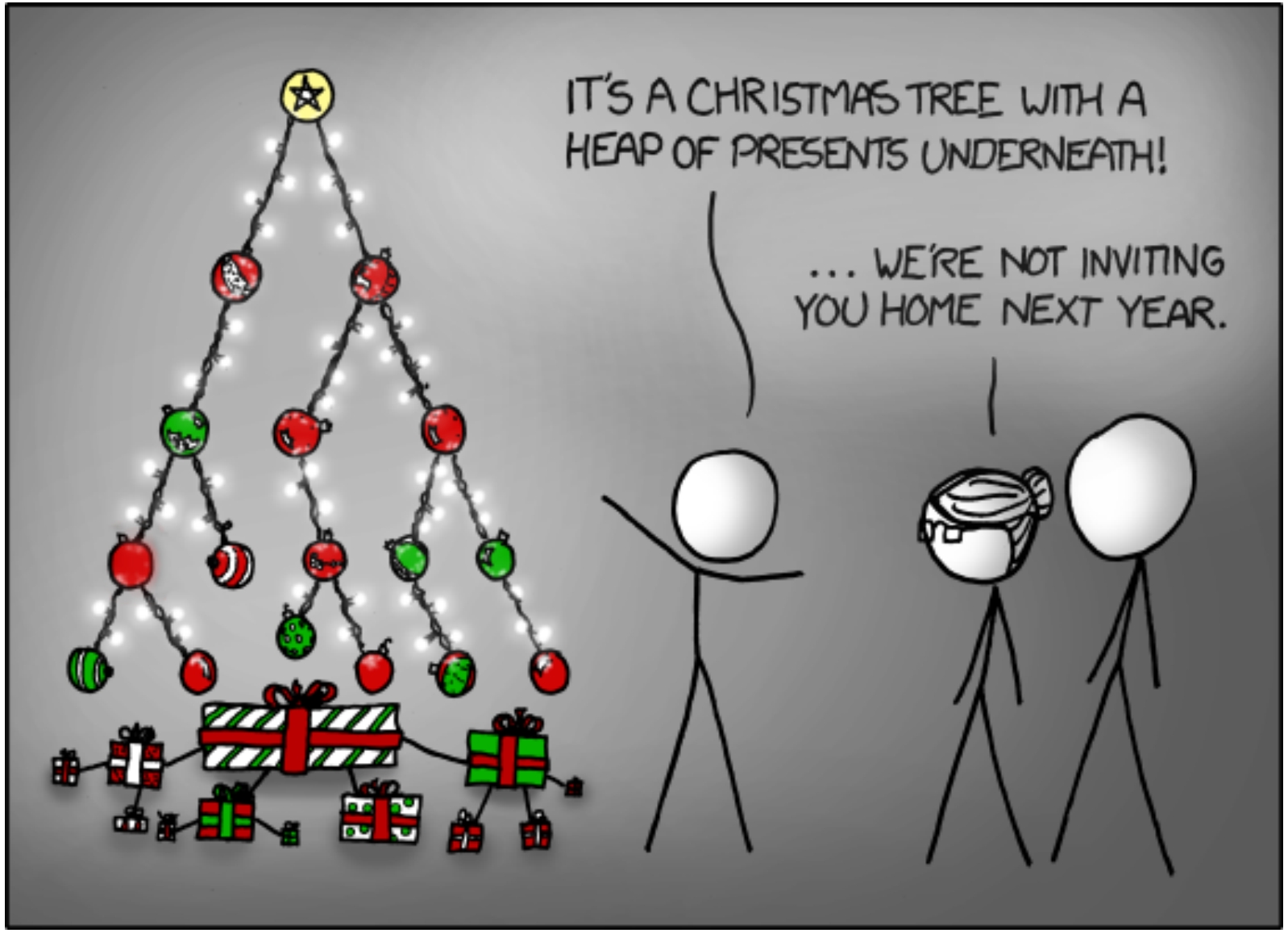
Data Compression



Navigation

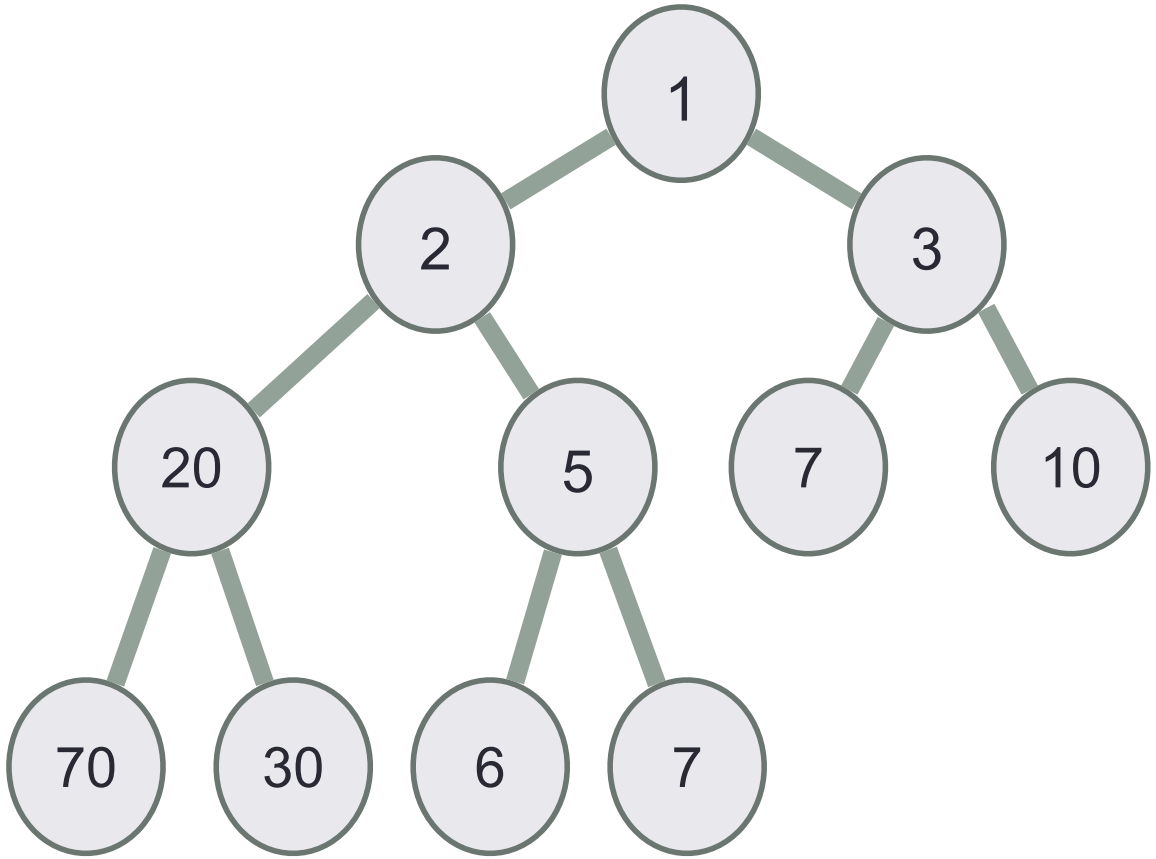


Network Design



Think of binary heap as a heap of presents!!

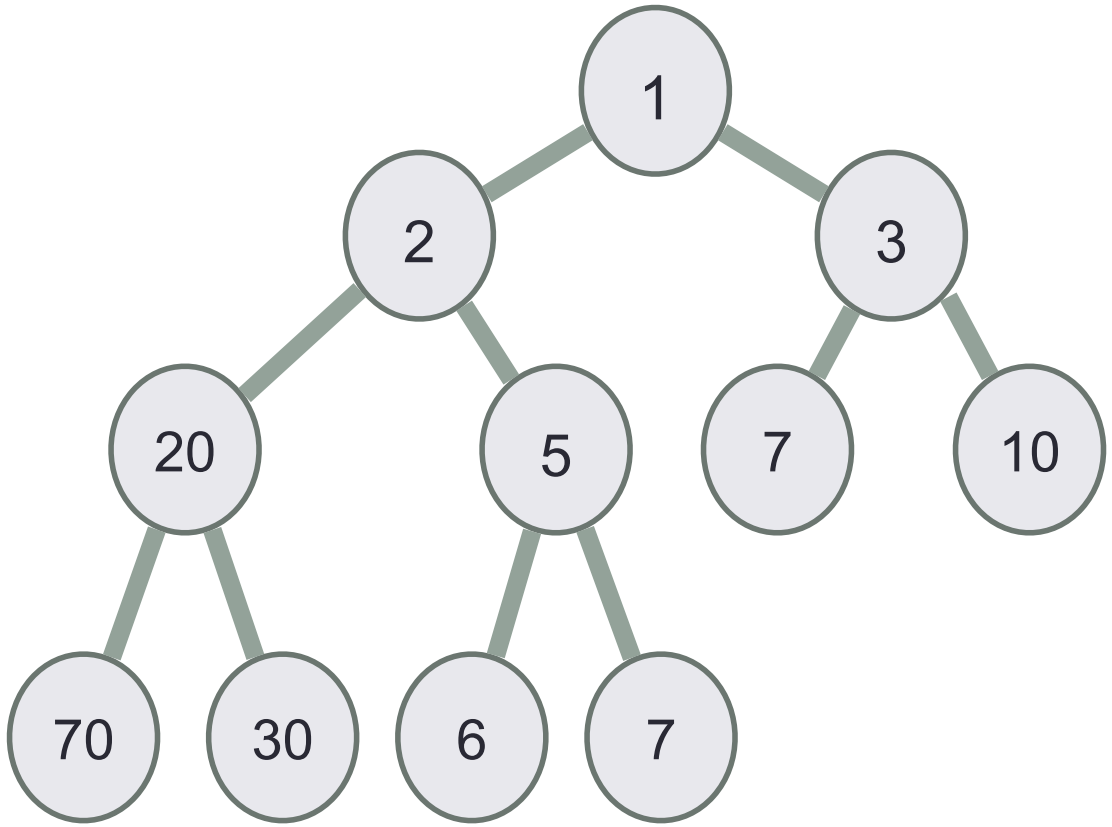
Two important properties of a binary heap tree



(1) Shape property:

(2) Heap property :

Two important properties of a binary heap tree



Example of a min-heap

(1) Shape property:

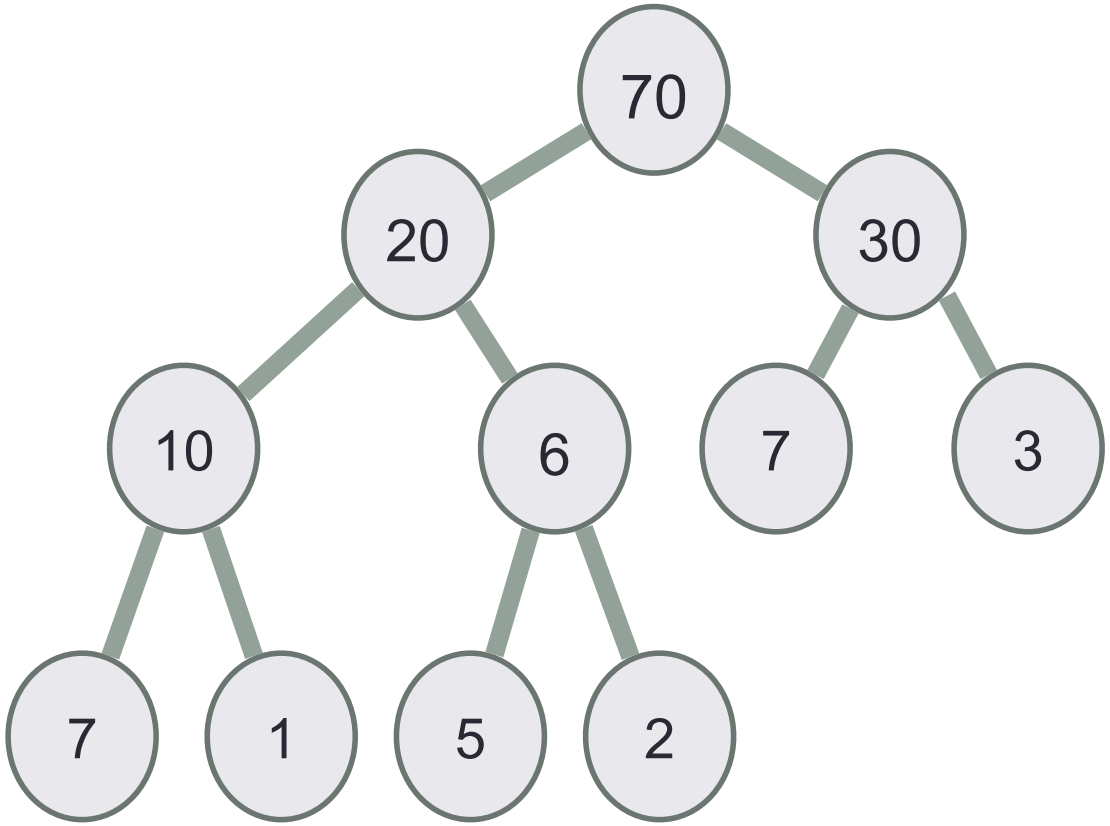
Internally, a heap is a **complete binary tree**

(2) Heap property:

In a min-heap, for each node (x):

key(x) <= key(children of x)

Two important properties of a binary heap tree



Example of a max-heap

(1) Shape property:

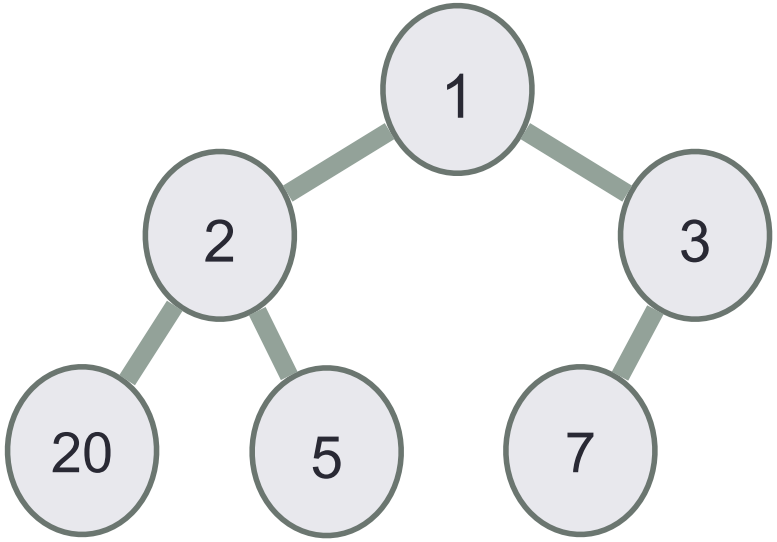
Internally, a heap is a **complete binary tree**

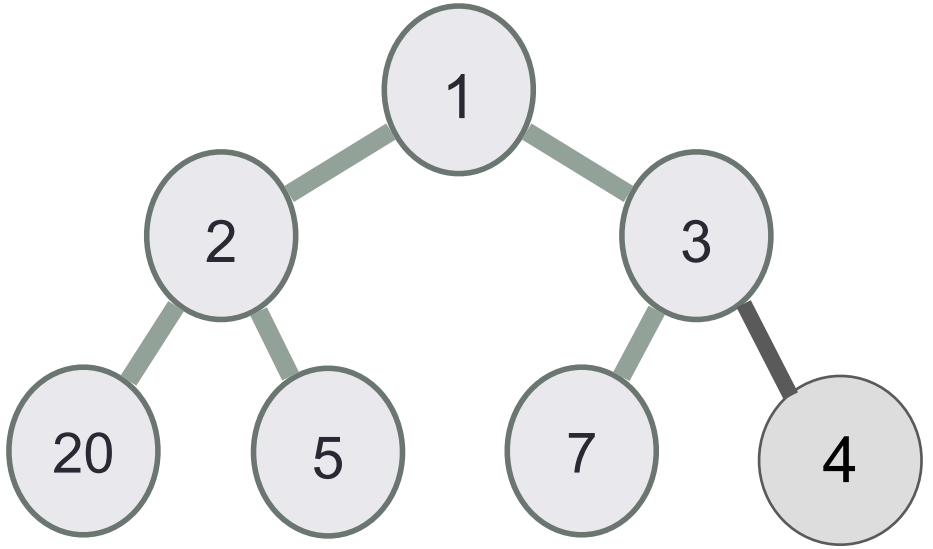
(2) Heap property:

In a max-heap, for each node (x):

key(x) >= key(children of x)

Complete Binary Tree





Complete Binary Tree:
Every level is completely filled (except possibly the last level), and all nodes on the last level are as far left as possible

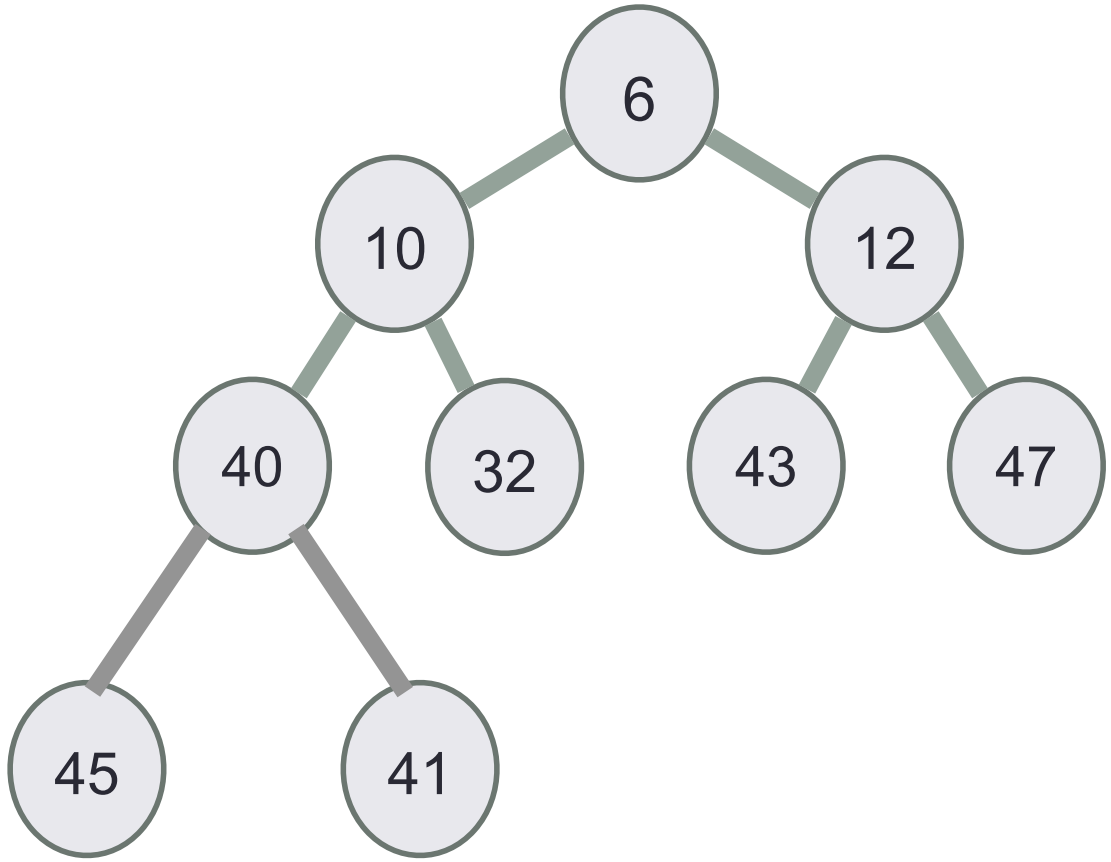
Full Binary Tree: A complete binary tree with last level completely filled

Complete/full binary trees are **balanced trees**!

Identifying heaps (iclicker)

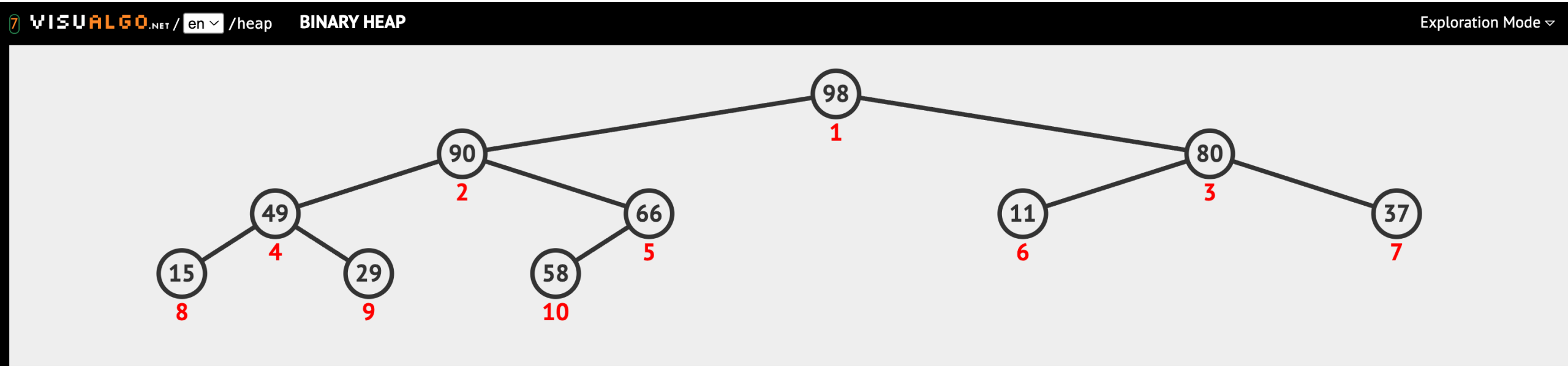
Starting with the following min-Heap which of the following operations will result in something that is NOT a min Heap

A. Swap the keys 40 and 32
B. Swap the keys 32 and 43
C. Swap the keys 43 and 40
D. Insert 50 as the left child of 45
E. C&D



Visualizing the operations of a heap

- VisualAlgo (Heap): https://visualgo.net/en/heap
- Heap Operations:
  - top(): get the root (either min or max key): O(1)
  - push(key): insert a key - O(log n)
  - pop(): remove the top element- O(log n)



```
procedure push(x: key value)
   insert x in the first open spot in the tree //O(1)
   while(x has a parent && parent(x) < x): //Bubble up O(logn)
        swap(x, parent(x))
   done
```

Push **the values** 5, 7, 1, 3, 2, 20 into an initially empty max-heap.
Show how keys bubble up on an insert to restore the heap property.

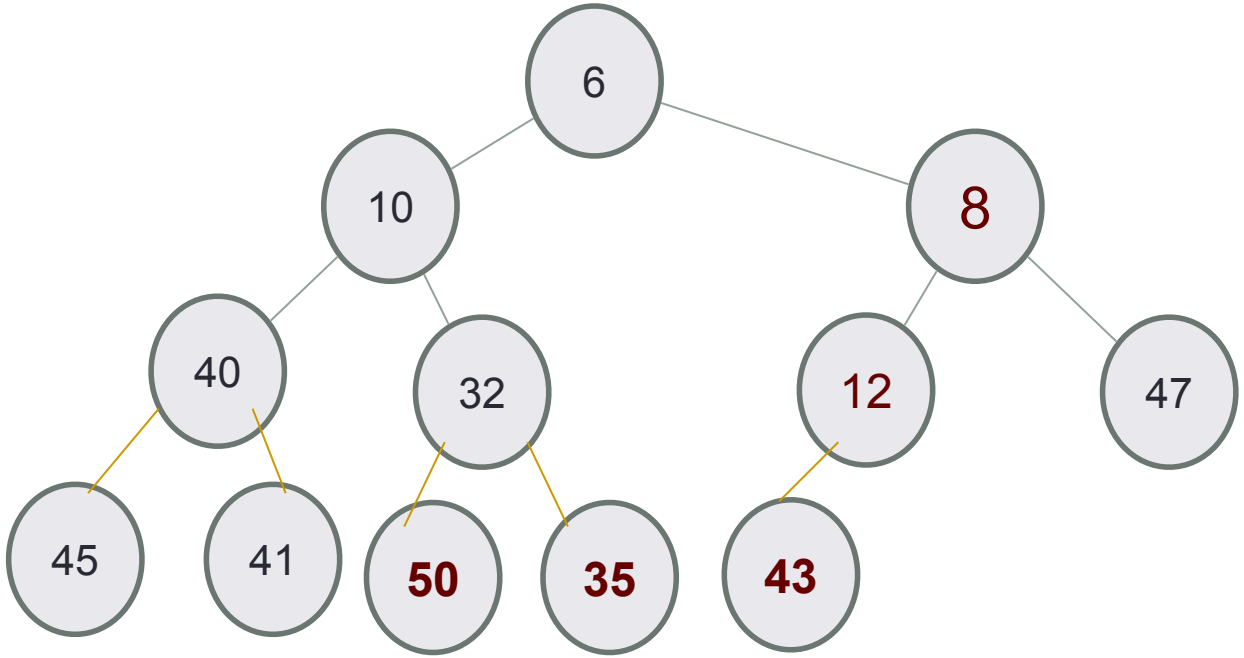
pop(): delete the key at the top()

procedure pop()

Step1: Replace the key of root with key of last node in the last level.

Step 2: Starting at the root “Bubble down” until the heap property is restored

return {the top element was deleted}



"Bubble down" in a min-heap:

- swap key of node with child that has the smallest key value until the heap property is restored

“Bubble down” in a max heap:

- swap key of node with child that has the largest key value until the heap property is restored

.

Final piece of the puzzle

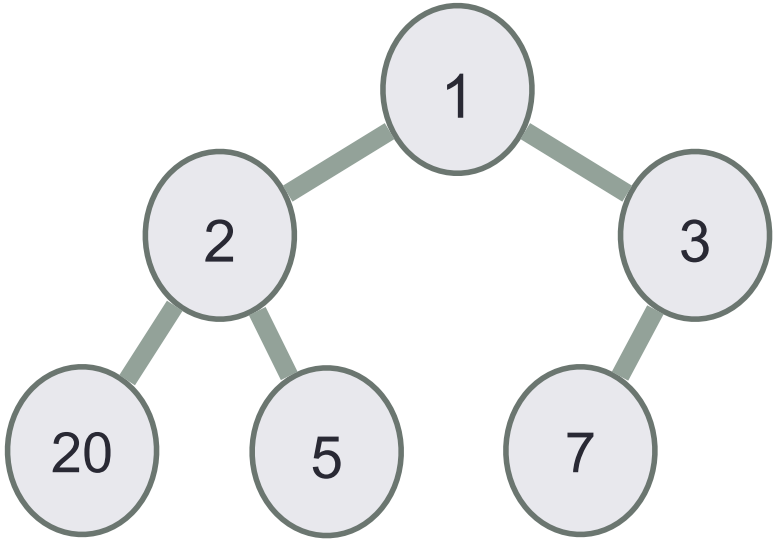
```
procedure push(x: key value)
   insert x in the first open spot in the tree //O(1)—— how?
   while(x has a parent && parent(x) < x): //Bubble up O(logn)
        swap(x, parent(x))
   done
```

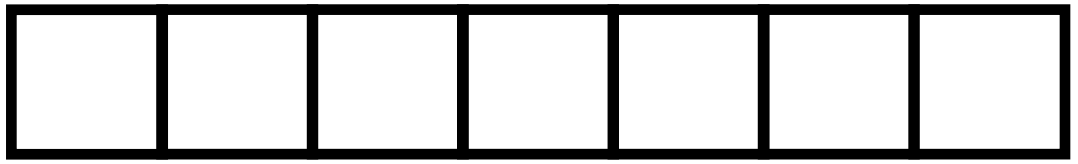
Under the hood the binary heap is represented as a vector!!

Inserting in the last open spot at the last level in the tree is essentially just a push_back() operation to the vector which is O(1)!

But then you need to know how to traverse up and down the binary heap "tree" using only the vector

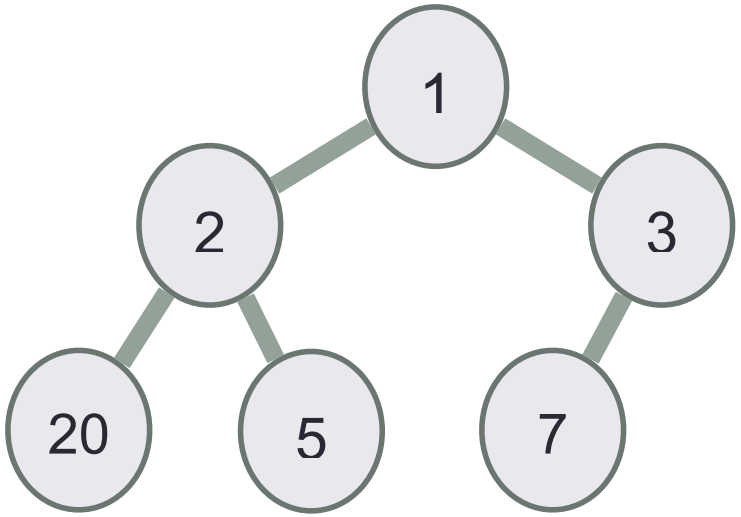
Representing a complete binary tree as a vector!





- How is the index of each key related to the index of its parent?
- How is the index of each key related to the indices of its left and right child?

Representing a complete binary tree as a vector!



index
key
parent
left child
right child

1	2	3	20	5	7	

Root is at index 0

For a key at index i, index of its

- **parent is** $\lfloor (i-1)/2 \rfloor$
- **left child is** $2i+1$
- **right child is** $2i+2$

Activity: For a key at index i, determine the indices of its parent and children.

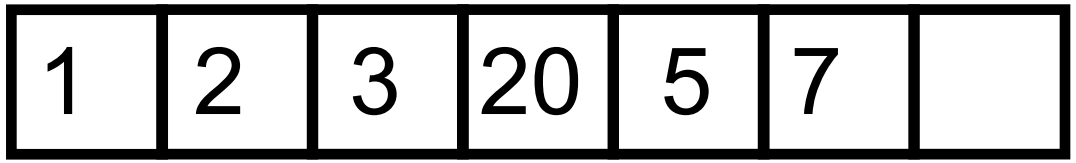
Look for a pattern to confirm the formulas on the left

Traverse up the tree using the vector (only)!

Root is at index 0

For a key at index i, index of its

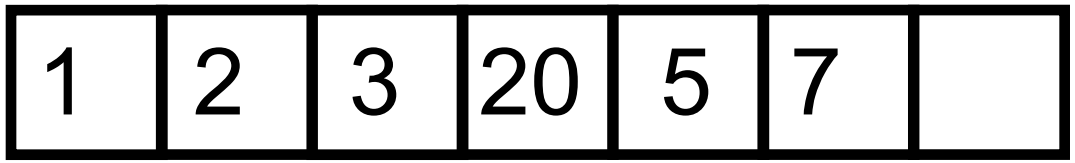
- **parent is** $\lfloor (i-1)/2 \rfloor$
- **left child is** $2i+1$
- **right child is** $2i+2$



Activity: Starting at the last node in the last level (7), write the indices of the keys visited on the path to the root node with key (1):

A. 5, 4, 3, 2, 1, 0
B. 5, 4, 2, 1, 0
C. 5, 3, 1
D. 5, 2, 0
E. None of the above

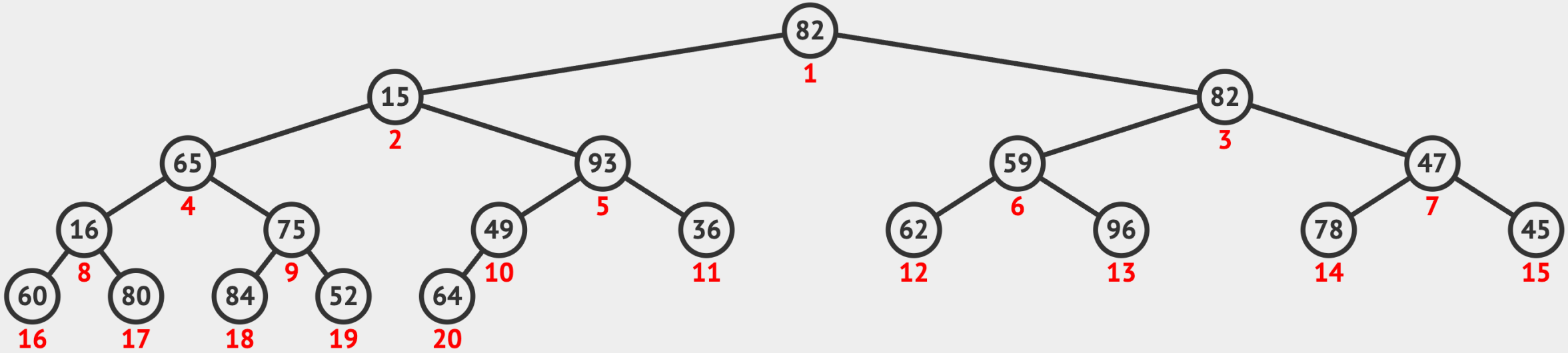
Run the push algo on the vector representation of the binary heap
push(4)



What is the complexity of adding new keys to a complete binary tree?

A. O(1)
B. O(log n)
C. O(n)
D. None of the above

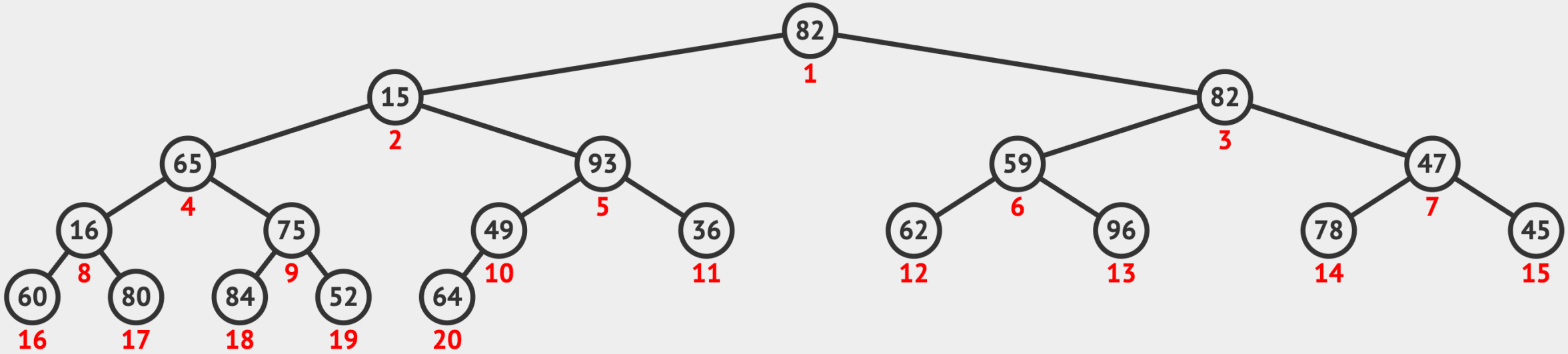
Heapify: Fast construction of a heap



Efficient Approach to Construct a Heap from vector with n keys:
Start from the last non-leaf node and work upwards to the root (index 0).
For each node, perform “bubble-down” to restore the heap property in its subtree.

Link to visualization

Heapify: Run Time Complexity




```
void buildHeap(vector<int>& v) {
    int n = v.size();
    // Start from last non-leaf node → root
    for (int i = n/2 - 1; i >= 0; i--) {
        bubbleDown(v, i);
    }
}
```